

# Step 1: Create the EKS Host Cluster

```
cat > cluster.yaml <<EOF
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: vcluster-host
  region: us-east-1

kubernetesNetworkConfig:
  serviceIPv4CIDR: "[IP_ADDRESS]"

autoModeConfig:
  enabled: true
EOF
```

```
eksctl create cluster -f cluster.yaml
```

What this does:

1. Creates a brand new EKS cluster called vcluster-host in us-east-1
2. Auto Mode enabled — AWS manages nodes, scaling, load balancing, storage, and DNS automatically. No need to install Karpenter, AWS Load Balancer Controller, or EBS CSI driver yourself
3. Service CIDR **10.96.0.0/12** — this is the internal IP range Kubernetes uses for Services. vCluster requires this specific range because it's the default CIDR vCluster uses internally. If they don't match, services inside virtual clusters won't route correctly to the host cluster
4. eksctl also creates a new VPC, subnets (2 public, 2 private), IAM roles, and security groups automatically

Think of this as building the house — the foundation that everything else runs on.

## Step 2: Apply IngressClass and StorageClass

```
kubectl apply -f - <<EOF
apiVersion: eks.amazonaws.com/v1
kind: IngressClassParams
metadata:
  name: alb
spec:
  scheme: internet-facing
  group:
    name: vcluster
---
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: alb
  annotations:
    ingressclass.kubernetes.io/is-default-class: "true"
spec:
  controller: eks.amazonaws.com/alb
  parameters:
    apiGroup: eks.amazonaws.com
    kind: IngressClassParams
    name: alb
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: auto-ebs-sc
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.eks.amazonaws.com
volumeBindingMode: WaitForFirstConsumer
parameters:
  type: gp3
  encrypted: "true"
EOF
```

What each piece does:

IngressClassParams (alb):

1. `scheme: internet-facing` — tells the ALB to be publicly accessible from the internet
2. `group.name: vcluster` — this is the key setting. It groups ALL Ingress objects that use this class into a single ALB. Without this, every Ingress would create its own separate ALB (expensive and wasteful). With it, one ALB serves all apps across all virtual clusters using path-based routing

IngressClass (alb):

1. `controller: eks.amazonaws.com/alb` — tells Kubernetes to use the EKS Auto Mode built-in ALB controller (not a self-installed one)
2. `is-default-class: "true"` — any Ingress that doesn't specify a class automatically uses this one
3. Links to the IngressClassParams above for the ALB configuration

StorageClass (auto-ebs-sc):

1. `provisioner: ebs.csi.eks.amazonaws.com` — uses the EKS Auto Mode built-in EBS CSI driver
2. `volumeBindingMode: WaitForFirstConsumer` — only creates EBS volumes when a pod actually needs one (saves money — no idle volumes)
3. `type: gp3` — uses gp3 EBS volumes (good balance of price and performance)
4. `encrypted: "true"` — all volumes are encrypted at rest (security best practice)
5. `is-default-class: "true"` — any PVC that doesn't specify a storage class uses this one

Think of this as setting up the house rules — how load balancing and storage should work for all tenants.

## Step 3: Deploy vCluster Platform via Helm

```
helm repo add vcluster https://charts.loft.sh
helm upgrade --install vcluster-pro vcluster/vcluster-platform \
  -n vcluster-poc --create-namespace --version 4.0.1 --values <(cat <<EOF
resources:
  limits:
    memory: 4Gi
  requests:
    cpu: "1"
    memory: 4Gi
replicaCount: 1
config:
  projectNamespacePrefix: loft-p-
audit:
  enabled: true
admin:
  create: true
  username: admin
  password: password
ingress:
  enabled: true
  name: loft-ingress
  annotations:
    alb.ingress.kubernetes.io/subnets: subnet-04dd294987985b0f4,subnet-0bf577d0b0a232af8
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/load-balancer-name: vcluster-alb
  ingressClass: alb
  path: /*
  tls:
    enabled: false
affinity:
  nodeAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
    preference:
      matchExpressions:
        - key: eks.amazonaws.com/compute-type
        operator: In
  values:
    - auto
EOF )
```

What each section does:

1. `helm repo add` — adds the vCluster Helm chart repository (where the installation package lives)
2. `helm upgrade --install` — installs vCluster platform (or upgrades if already installed)
3. `-n vcluster-poc --create-namespace` — installs into a new namespace called vcluster-poc
4. `--version 4.0.1` — specific version of the vCluster platform

Resources:

The vCluster platform pod gets 1 CPU and 4Gi memory — this runs the management UI and API

Config:

1. `projectNamespacePrefix: loft-p-` — namespaces for vCluster projects will be prefixed with loft-p-
2. `audit.enabled: true` — tracks who did what (important for compliance)

Admin:

Creates an initial admin user with username admin and password password

Ingress:

1. `subnets` — two public subnets where the ALB gets deployed (must be in the same VPC as the cluster)
2. `scheme: internet-facing` — ALB is publicly accessible
3. `load-balancer-name: vcluster-alb` — names the ALB so we can reference it later
4. `ingressClass: alb` — uses our IngressClass from Step 2
5. `path: /*` — routes ALL traffic to the vCluster UI
6. `tls: false` — no HTTPS (since we skipped ACM/Route 53)

Affinity:

`eks.amazonaws.com/compute-type: auto` — prefers to schedule the vCluster pod on nodes managed by EKS Auto Mode

Think of this as installing the building management system — the control panel where tenants (QA teams) can create their own spaces.

Step 4: Fix the Ingress Host

```
kubectl patch ingress loft-ingress -n vcluster-poc --type='json' -p='[{"op": "remove", "path": "/spec/rules/0/host"}]'
```

What this does:

1. The Helm chart injected a default host `loft.mydomain.tld` into the Ingress rules
2. This means the ALB would ONLY route traffic that comes with that hostname in the request
3. Since we're accessing via the raw ALB DNS name (`vcluster-alb-496573325.us-east-1.elb.amazonaws.com`), the host doesn't match → 404
4. Removing the host makes the Ingress accept traffic from any hostname → the ALB routes everything to vCluster UI

Think of this like removing the name tag from the doorbell — now anyone can ring it, not just someone who knows the specific name.

## Step 5: Create Two Virtual Clusters via the UI

Logged into the vCluster UI at <http://vcluster-alb-496573325.us-east-1.elb.amazonaws.com> and created two virtual clusters with this config:

```
sync:
  fromHost:
    ingressClasses:
      enabled: true
    storageClasses:
      enabled: true
  toHost:
    ingresses:
      enabled: true
controlPlane:
  coredns:
    enabled: true
  embedded: true
```

What each setting does:

1. `fromHost.ingressClasses.enabled` — copies the alb IngressClass FROM the host cluster INTO the virtual cluster. Without this, apps in the virtual cluster wouldn't know how to create ALBs
2. `fromHost.storageClasses.enabled` — copies the auto-ebs-sc StorageClass FROM the host INTO the virtual cluster. Without this, PVCs inside the virtual cluster would fail because there's no storage class
3. `toHost.ingresses.enabled` — syncs Ingress objects created inside the virtual cluster BACK TO the host cluster. This is critical — when you create an Ingress in a virtual cluster, vCluster copies it to the host, where the ALB controller picks it up and adds a new routing rule
4. `coredns.embedded: true` — runs a lightweight DNS server inside the virtual cluster control plane so apps can resolve service names (e.g., `echoserver.echoserver.svc.cluster.local`)

Think of this like creating two apartments in the building — each apartment has its own entrance (API server) but shares the building's plumbing (load balancer) and electricity (storage).

Step 6: Connect to Virtual Clusters and Deploy Apps

Connect using vCluster CLI:

```
vcluster connect vcluster-bftww -n loft-default-v-vcluster-bftww
```

What this does:

1. Creates a port-forward tunnel from your EC2 instance directly to the virtual cluster's API server (a pod inside the host cluster)
2. Updates your kubeconfig so kubectl commands go to the virtual cluster instead of the host
3. Must stay running in one terminal — use a second terminal for kubectl commands



Deploy app1 to virtual cluster 1:

```
kubect! apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: echoserver
---
apiVersion: v1
kind: Service
metadata:
  name: echoserver
  namespace: echoserver
spec:
  ports:
    - port: 80
      targetPort: 8080
      protocol: TCP
  type: NodePort
  selector:
    app: echoserver
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: echoserver
  namespace: echoserver
spec:
  selector:
    matchLabels:
      app: echoserver
  replicas: 1
  template:
    metadata:
      labels:
        app: echoserver
    spec:
      containers:
        - image: registry.k8s.io/e2e-test-images/echoserver:2.5
          imagePullPolicy: Always
          name: echoserver
          ports:
            - containerPort: 8080
          volumeMounts:
            - name: ebs-volume
              mountPath: /mnt/data
```

```
volumes:
  - name: ebs-volume
    persistentVolumeClaim:
      claimName: ebs-claim
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: echoserver
  namespace: echoserver
  annotations:
    alb.ingress.kubernetes.io/load-balancer-name: vcluster-alb
    alb.ingress.kubernetes.io/subnets: subnet-04dd294987985b0f4,subnet-0bf577d0b0a232af8
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/group.order: "-999"
spec:
  ingressClassName: alb
  rules:
    - host: vcluster-alb-496573325.us-east-1.elb.amazonaws.com
      http:
        paths:
          - path: /app1
            pathType: ImplementationSpecific
            backend:
              service:
                name: echoserver
                port:
                  number: 80
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: ebs-claim
  namespace: echoserver
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: auto-ebs-sc
EOF
```

What each resource does:

1. Namespace (echoserver) — creates an isolated namespace for the app inside the virtual cluster
2. Service — exposes the echoserver pod on port 80, forwards to container port 8080. NodePort type makes it accessible to the ALB
3. Deployment — runs 1 replica of the echoserver container (an nginx-based echo service that returns request info). Mounts an EBS volume at /mnt/data
4. Ingress — this is where the magic happens:
  - load-balancer-name: vcluster-alb — joins the SAME ALB as the vCluster UI and other apps
  - subnets — your public subnets
  - group.order: "-999" — sets routing priority (lower = last). The vCluster UI's /\* path takes lower priority, so /app1 matches first
  - path: /app1 — path-based routing. Requests to /app1 go to this echoserver
  - vCluster syncs this Ingress to the host cluster → ALB controller picks it up → adds a new rule
5. PVC — requests 1Gi of encrypted gp3 EBS storage using our auto-ebs-sc StorageClass. The volume is only created when the pod gets scheduled (WaitForFirstConsumer)

Then repeated for virtual cluster 2 with /app2 path instead of /app1.

The Complete Flow (How Traffic Reaches Your Apps)

```
aws elbv2 describe-load-balancers --names vcluster-alb --region us-east-1 --query 'LoadBalancers.DNSName' --output text
```

User browser

- `http://vcluster-alb-<account>.us-east-1.elb.amazonaws.com/app1`
- ALB checks path rules
- /app1 matches → routes to echoserver Service in virtual cluster 1
- /app2 matches → routes to echoserver Service in virtual cluster 2
- /\* matches → routes to vCluster UI

All through one single ALB, with each virtual cluster completely isolated from the other.